MODULE

19

Performance & internals

How Python stores your data, how to find what's slow, and how to fix it: the internals interviewers ask about.

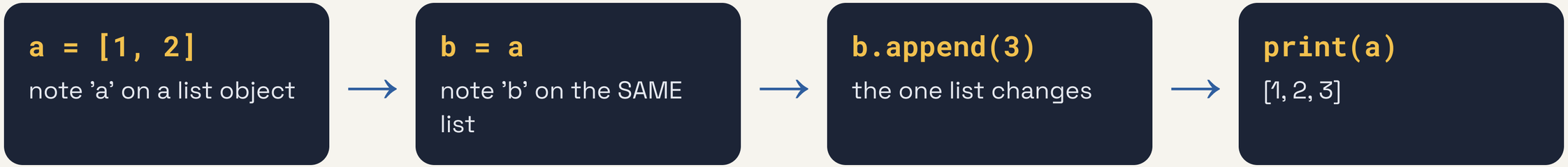
YOU WILL LEARN

The memory model	`01_memory_model.py`
Measure first	`02_measure_first.py`
Speed tricks that matter	`03_speed_tricks.py`
Saving memory	`04_memory_tricks.py`
Bytecode, weakref, NumPy	`05_bytecode_and_numpy.py`
Interview Q&A	`INTERVIEW.md`

PICTURE IT MEMORY MODEL

Names are labels, not boxes

🏷️ A variable is a **sticky note** on an object, not a box that holds it. **b = a** puts a second note on the **same** object. Changing the object through one note shows up through the other.



TRY IT MEMORY MODEL

References, ids and reference counts

```python
import sys

a = [1, 2]
b = a                          # same object
c = [1, 2]                     # equal, different object
b.append(3)
print(a, a is b, a == c, a is c)
print(id(a) == id(b))

print(sys.getrefcount(a) - 1, "references to the list")   # a, b
del b
print(sys.getrefcount(a) - 1, "reference after del b")

def add_item(lst):             # functions get the SAME object
    lst.append("x")
add_item(a)
print(a)
```

OUTPUT

```
[1, 2, 3] True False False
True
2 references to the list
1 reference after del b
[1, 2, 3, 'x']
```

1. Python passes **references to objects** (”call by sharing”).
2. Each object counts its references; at 0 it’s freed at once.
3. Cycles are cleaned up by the **garbage collector** (**gc**).

WHY MEASURE FIRST

Don’t guess where it’s slow

THE PROBLEM

A teammate rewrote a function with clever tricks for 2 days. The page was still slow: 95% of the time was a **database query in a loop**.

THE IDEA

Measure first: timeit for small snippets, cProfile to find the slow function, then fix the biggest thing.

TRY IT MEASURE FIRST

cProfile finds the hot spot

```python
import cProfile, pstats, time

def load_user(uid):  time.sleep(0.01); return {"id": uid}
def format_row(u):   return f"{u['id']:>5}"

def report():
    return [format_row(load_user(i)) for i in range(30)]   # N slow calls!

prof = cProfile.Profile()
prof.runcall(report)
stats = pstats.Stats(prof).stats          # {(file, line, name): (..., ncalls, tottime, cumtime, ...)}
print(f"{'function':12} {'ncalls':>6} {'cumtime':>8}")
for (_, _, name), (_, ncalls, _, cumtime, _) in stats.items():
    if name in ("load_user", "format_row"):
        print(f"{name:12} {ncalls:>6} {cumtime:>7.2f}s")
```

OUTPUT

```
function     ncalls  cumtime
load_user        30    0.31s
format_row       30    0.00s
```

1. **ncalls**: how often; **cumtime**: total time spent inside.
2. **load_user** eats the time: batch it into one query.
3. In web apps: also look at the **number of SQL queries** per request.

STEP BY STEP SPEED TRICKS

Fixes that actually matter (biggest first)

FIX	TYPICAL GAIN	EXAMPLE
Better algorithm / data structure	10–10,000×	list lookup → set; nested loops → dict
Fewer I/O round trips	10–100×	1 query with JOIN instead of N queries
Caching	huge for repeats	`lru_cache`, Redis (Module 26)
Built-ins and C libraries	2–100×	`sum()`, `sorted()`, NumPy, pandas
Concurrency	N× for waiting	asyncio / threads (Module 17)
Micro-tricks (local variables...)	a few %	only in proven hot loops

TRY IT SPEED TRICKS

Built-ins and NumPy vs plain loops

```python
import timeit
import numpy as np

nums = list(range(1_000_000))
arr = np.arange(1_000_000)

def loop_sum():
    total = 0
    for n in nums:
        total += n * n
    return total

t_loop = timeit.timeit(loop_sum, number=5) / 5
t_gen = timeit.timeit(lambda: sum(n * n for n in nums), number=5) / 5
t_np = timeit.timeit(lambda: int((arr * arr).sum()), number=5) / 5
print(f"for loop : {t_loop * 1000:6.1f} ms")
print(f"sum(gen) : {t_gen * 1000:6.1f} ms")
print(f"numpy    : {t_np * 1000:6.1f} ms  (~{t_loop / t_np:.0f}x faster)")
```

OUTPUT

```
for loop :   79.6 ms
sum(gen) :  160.5 ms
numpy    :    8.4 ms  (~10x faster)
```

1. NumPy runs the loop in **C** on a compact array.
2. Use it for numbers; pandas for tables.
3. Numbers differ per machine; the ranking doesn't.

TRY IT SAVING MEMORY

__slots__ and generators

```
import sys, tracemalloc

class Point:
    def __init__(self, x, y): self.x, self.y = x, y

class SlimPoint:
    __slots__ = ("x", "y")                # no per-object __dict__
    def __init__(self, x, y): self.x, self.y = x, y

for cls in (Point, SlimPoint):
    tracemalloc.start()
    pts = [cls(i, i) for i in range(100_000)]
    size, _ = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    print(f"{cls.__name__:9} {size / 1e6:5.1f} MB")

print(sys.getsizeof([n for n in range(100_000)]), "bytes as a list")
print(sys.getsizeof(n for n in range(100_000)), "bytes as a generator")
```

OUTPUT

```
Point      12.0 MB
SlimPoint   8.8 MB
800984 bytes as a list
192 bytes as a generator
```

1. **__slots__** saves memory with millions of small objects.
2. **Generators** keep memory flat (Module 11).
3. **tracemalloc** measures what your code allocates.

COMMON MISTAKE PERFORMANCE

Premature optimisation

✗ WRONG

```
# "faster" but nobody can read it
r=[*map(lambda t:t[0]*t[1],zip(p,q))]
```

Unreadable code for an unmeasured gain. Bugs hide in it, and the real bottleneck (I/O) is still there.

✓ RIGHT

```
totals = [price * qty for price, qty in zip(prices, quantities)]
# then: measure. Is THIS the slow part?
```

Write clear code, **measure**, then optimise the hot spot. ”Premature optimisation is the root of all evil.”

IN REAL JOBS PERFORMANCE

Performance work in web jobs

Find N+1 queries

```
Order.objects.select_related("user")
             .prefetch_related("items")
```

Profile one slow endpoint

```
python -m cProfile -o out.prof app.py
snakeviz out.prof
```

Cache expensive reads

```
@cache.memoize(timeout=300)
def exchange_rates(): ...
```

Move heavy work off the request

```
generate_pdf.delay(invoice_id)  # Celery
```

INTERVIEW SAY THIS IN THE INTERVIEW

“How does Python manage memory?”

- Everything is an **object** on a private heap; variables are **references** (names) to objects.
- CPython uses **reference counting**: when an object’s count drops to 0 it’s freed immediately.
- A cyclic **garbage collector** (**gc**, 3 generations) cleans up reference cycles.
- Small objects come from the **pymalloc** allocator; small ints and some strings are cached (interned).
- Arguments are passed by **object reference**: mutating a list inside a function changes the caller’s list.

> **Follow-up they ask next:** ”What is a memory leak in Python?” (objects still referenced by mistake: global caches, lists that only grow, cycles with __del__; find them with tracemalloc).

INTERVIEW SAY THIS IN THE INTERVIEW

“Your API endpoint is slow. How do you find and fix it?”

- **Measure**: timings in logs/APM, then profile the endpoint (cProfile, py-spy).
- Check the **database** first: count queries (N+1?), EXPLAIN, missing indexes.
- Check **external calls**: add timeouts, run them concurrently, cache results.
- Then the code: better data structures and algorithms, fewer copies.
- Move slow work to **background jobs**; add **caching** (Redis) and pagination.

> **Follow-up they ask next:** ”What is __slots__ and when would you use it?” (fixed attributes, no __dict__: less memory for millions of instances, slightly faster attribute access).

RECAP REMEMBER THESE

Module 19 in 6 lines

1. Variables are **labels**; **b = a** shares the object.
2. Reference counting frees objects at 0; **gc** handles cycles.
3. **Measure first**: timeit, cProfile, tracemalloc.
4. Biggest wins: algorithm, fewer I/O round trips, caching.
5. Built-ins and **NumPy** push loops into C.
6. **__slots__** and generators save memory.

PRACTICE NOW

Do exercises/ex19_make_it_fast.py: make a slow report 100x faster without changing the result.